Java Masterclass for Beginners

Subtitle: *Learn Java from Scratch | OOP | Collections | Multithreading | JDBC | File Handling | Build Real-World Projects*

Chapter 1: Introduction to Java

- **Learning Objective:** Provide a solid foundation in Java by understanding what Java is, its history, editions, and ecosystem. Set up the development environment (JDK, IDEs) and write the first Java program.
- **Skills Covered:** Understanding of Java platform, "Write Once Run Anywhere" principle, JVM/JRE/JDK roles, installing and configuring Java SDK, using IDEs (VS Code, IntelliJ, Eclipse), writing and compiling a Java program.
- **Prerequisites:** None (Absolute beginners). Basic computer usage knowledge.
- **Estimated Learning Time:** 6–8 hours.
- **Number of Lessons:** 6

*Lesson 1.1: **What is Java? Introduction & History***

- **Difficulty Level:** Beginner
- **Learning Outcome:** Explain Java's purpose, evolution, and key features (platform-independence, OOP focus). Understand the "Write Once, Run Anywhere" (WORA) concept.
- **Topics Covered:**
- Definition of Java, uses (enterprise, Android, etc.)
- Java versions timeline (Oak → Java 1.0 to latest)
- Key features (object-oriented, platform-independent, robust)【20†L41-L50】.
- **Practical Exercises:**
- Research and list 3 popular applications built with Java.
- Identify differences between Java and other languages you know.
- **Mini Assignment:** Write a short essay or presentation on how Java evolved from 1995 to present.
- **Common Mistakes:** Confusing Java with JavaScript; expecting immediate output without compiling.
- **Interview Questions:** "What is Java and what does 'platform-independent' mean?"
- **Quiz Questions:** "Java was originally called ____." (Answer: Oak)

*Lesson 1.2: **Java Editions and Platform Components (JVM, JRE, JDK)***

- **Difficulty Level:** Beginner

- **Learning Outcome:** Distinguish between Java SE, EE (Jakarta EE), ME editions, and explain roles of JVM, JRE, and JDK.
- **Topics Covered:**
- Java SE vs Enterprise vs Micro editions
- Java Virtual Machine (JVM) role【20†L81-L89】
- Java Runtime Environment (JRE) and Java Development Kit (JDK) differences【20†L147-L152】
- "javac" compiler and "java" interpreter.
- **Practical Exercises:**
- Examine your system: find the path of `java` and `javac`.
- Use `java -version` and `javac -version` in command-line.
- **Mini Assignment:** Create a diagram showing Java application flow from source code to execution (including JVM and JRE).
- **Common Mistakes:** Mixing up JDK and JRE, running a program without compiling it first.
- **Interview Questions:** "Explain JVM, JRE, and JDK."
- **Quiz Questions:** "Which component contains the Java compiler? (a) JVM (b) JRE (c) JDK" (Answer: c)

Lesson 1.3: ***Installing Java and Command-Line Hello World***

- **Difficulty Level:** Beginner
- **Learning Outcome:** Install Java Development Kit, set environment variables, write and run a simple Java program using the command line.
- **Topics Covered:**
- Downloading and installing the latest JDK (LTS version Java 25)
- Setting PATH and JAVA_HOME on Windows/Mac/Linux
- Creating `HelloWorld.java`, compiling (`javac`), and running (`java`)
- **Practical Exercises:**
- Install JDK on your system. Verify installation via command-line.
- Write a "Hello, World!" program and run it.
- **Mini Assignment:** Modify the Hello World program to read your name as input and greet you.
- **Common Mistakes:** Forgetting to compile before running; wrong class filename vs public class name; PATH not set.
- **Interview Questions:** "How do you compile and run a Java program from the command line?"
- **Quiz Questions:** "What command compiles a Java source file? (a) java (b) javac (c) jdk" (Answer: b)

Lesson 1.4: ***IDE Setup (VS Code, IntelliJ IDEA, Eclipse)***

- **Difficulty Level:** Beginner
- **Learning Outcome:** Set up a Java development environment in popular IDEs. Create and run Java projects in VS Code, IntelliJ IDEA, and Eclipse.
- **Topics Covered:**
- Installing and configuring VS Code with Java extensions
- Installing IntelliJ IDEA Community Edition
- Creating a Java project and running programs in each IDE
- Project structure (src folder, package organization)
- **Practical Exercises:**
- Create a new Java project in VS Code; write and run a simple program.
- Do the same in IntelliJ IDEA and note differences.
- **Mini Assignment:** Write a small Java program (e.g., adding two numbers) in all three IDEs.
- **Common Mistakes:** Placing code outside the `main` method; forgetting to save files before running.
- **Interview Questions:** "Why use an IDE for Java development? Name popular Java IDEs."
- **Quiz Questions:** "Which file extension is used for Java source files? (a) .class (b) .java (c) .exe" (Answer: b)

Lesson 1.5: ***Compilation Process and Java Architecture***

- **Difficulty Level:** Beginner
- **Learning Outcome:** Understand Java's compile-run process and the architecture (Class Loader, Bytecode, JVM memory areas).
- **Topics Covered:**
- How `javac` produces bytecode (.class) and how JVM interprets/just-in-time compiles it
- Java Memory Model basics: Heap, Stack, Method Area, etc. 【20†L114-L123】
- Overview of Java platform architecture and virtual machine
- **Practical Exercises:**
- Use `javap` (Java disassembler) on a compiled class to see bytecode.
- Experiment with running a program, then open Task Manager/Activity Monitor to locate the Java process.
- **Mini Assignment:** Diagram the journey of Java code from writing to execution including compiler and JVM.
- **Common Mistakes:** Assuming Java is interpreted line-by-line; confusion about .java vs .class files.
- **Interview Questions:** "What does JVM stand for and what is its role?"

- **Quiz Questions:** “Java code is compiled into ___ before execution.” (Answer: bytecode)

Lesson 1.6: ***Java Language Foundations (Keywords, Naming Conventions)***

- **Difficulty Level:** Beginner
- **Learning Outcome:** Identify Java reserved keywords and apply standard naming conventions for classes, methods, variables, and packages.
- **Topics Covered:**
- Java reserved keywords (e.g. class, static, public) and their uses【6†L625-L632】
- Naming conventions: Class names (PascalCase), methods/variables (camelCase), constants (UPPER_SNAKE_CASE)
- Package naming conventions (reverse domain names)
- **Practical Exercises:**
- Review a Java program and highlight all keywords.
- Rename variables to follow Java naming style and compare readability.
- **Mini Assignment:** Refactor a given poorly-named Java class (with bad names) to use proper conventions.
- **Common Mistakes:** Using reserved keywords as identifiers; starting class names with lowercase.
- **Interview Questions:** “Give an example of a Java keyword and explain its use.”
- **Quiz Questions:** “According to Java conventions, class names should be written in: (a) camelCase (b) PascalCase (c) UPPERCASE” (Answer: b)

Chapter 2: Java Basics (Variables, Data Types, Operators, I/O)

- **Learning Objective:** Master Java’s basic syntax and constructs: declaring variables, using data types, applying operators, handling input/output, and understanding type conversions.
- **Skills Covered:** Variable declaration and initialization, understanding primitive and reference data types, using arithmetic/logical operators, performing console I/O, type casting, and writing comments.
- **Prerequisites:** Completion of Chapter 1 (Introduction to Java).
- **Estimated Learning Time:** 6–8 hours.
- **Number of Lessons:** 6

Lesson 2.1: ***Variables and Data Types***

- **Difficulty Level:** Beginner
- **Learning Outcome:** Declare and initialize variables of various types; understand primitive types vs reference types (String, etc).
- **Topics Covered:**

- Primitive data types: byte, short, int, long, float, double, char, boolean【6†L623-L632】
- Reference types: String (overview), arrays (intro)
- Variable declaration, initialization, and scope (local vs class-level)
- **Practical Exercises:**
- Declare variables of each primitive type and print their default values.
- Write a program that calculates and prints the area of a rectangle using `int` and `double` types.
- **Mini Assignment:** Create a Java program that uses multiple data types (int, float, char, boolean) and performs simple operations.
- **Common Mistakes:** Using uninitialized variables; overflow by exceeding type limits; confusing `char` and `String`.
- **Interview Questions:** "What is the difference between primitive types and reference types in Java?"
- **Quiz Questions:** "Which of these is NOT a primitive type? (a) String (b) int (c) boolean" (Answer: a)

Lesson 2.2: ***Operators and Expressions***

- **Difficulty Level:** Beginner
- **Learning Outcome:** Apply Java operators in expressions: arithmetic, relational, logical, assignment, and ternary.
- **Topics Covered:**
- Arithmetic operators (`+ - * / %`), operator precedence
- Relational (`==, !=, >, <, >=, <=`) and logical (`&&, ||, !`) operators
- Compound assignment (`+=, *=, ...`) and increment/decrement (`++, --`)
- Ternary operator `?:`
- **Practical Exercises:**
- Write expressions to compute a BMI (body mass index) and check if it falls in a healthy range.
- Demonstrate prefix vs postfix increment on integers with print statements.
- **Mini Assignment:** Write a calculator program using `Scanner` that can perform basic arithmetic based on user input of two numbers and an operator.
- **Common Mistakes:** Using == instead of = for assignment; forgetting operator precedence (e.g., mixing + and * without parentheses).
- **Interview Questions:** "What is the result of the expression `5 / 2` in Java and why?" (Answer: 2, because of integer division).
- **Quiz Questions:** "Which operator is used for logical OR in Java? (a) & (b) | (c) ||" (Answer: c)

Lesson 2.3: ***Console Input and Output***

- **Difficulty Level:** Beginner

- **Learning Outcome:** Read user input from the console and display output using standard streams.
- **Topics Covered:**
- `System.out.print` vs `println` vs `printf`
- Using `Scanner` class for input (nextInt, nextLine, etc.)
- Handling nextLine after nextInt issue (Scanner quirk)
- Formatted output with `printf` and String formatting
- **Practical Exercises:**
- Create a program that asks for the user's name and age, then prints a greeting.
- Use `printf` to format a table of values (e.g., multiplication table).
- **Mini Assignment:** Develop a unit converter (e.g., miles to kilometers) that takes input from the user.
- **Common Mistakes:** Not closing `Scanner`; input mismatch exceptions; mixing nextLine with nextInt without consuming newline.
- **Interview Questions:** "How do you read a line of input from the user in Java?"
- **Quiz Questions:** "Which class is commonly used for reading input in Java from the console? (a) ConsoleInput (b) Scanner (c) InputReader" (Answer: b)

Lesson 2.4: ***Type Casting and Conversion***

- **Difficulty Level:** Beginner
- **Learning Outcome:** Perform explicit and implicit type conversions; understand when casting is necessary.
- **Topics Covered:**
- Implicit (widening) conversions (e.g., int to double)
- Explicit (narrowing) conversions with `(type)` casting
- Casting primitives and potential data loss
- Converting between strings and numbers (Integer.parseInt, Double.valueOf, etc.)
- **Practical Exercises:**
- Write code that converts a `double` to `int` and observe truncation.
- Convert user-entered string to number and perform arithmetic.
- **Mini Assignment:** Build a program that reads two numbers as strings, converts them to `double`, multiplies them, and prints the result.
- **Common Mistakes:** Overflows when casting large types; forgetting to cast when assigning larger type to smaller type.
- **Interview Questions:** "What happens if you cast a `double` with value 3.99 to `int`?" (Answer: It becomes 3).
- **Quiz Questions:** "Which casting is implicit? (a) double to int (b) int to double" (Answer: b)

Lesson 2.5: ***Comments and Documentation***

- **Difficulty Level:** Beginner
- **Learning Outcome:** Use comments to document code. Differentiate between single-line, multi-line, and Javadoc comments.
- **Topics Covered:**
- Single-line (`//`) and multi-line (`/* ... */`) comments
- Javadoc comments (`/** ... */`) for methods and classes
- Importance of commenting and code readability
- **Practical Exercises:**
- Add comments to previous assignments explaining what each section of code does.
- Write Javadoc for a sample method you create (e.g., a function that adds two numbers).
- **Mini Assignment:** Create a small Java class (e.g., `Calculator`) and document it thoroughly with Javadoc.
- **Common Mistakes:** Leaving large blocks of commented-out code; forgetting to update comments when code changes.
- **Interview Questions:** "What is Javadoc and how is it used?"
- **Quiz Questions:** "Which comment style in Java is used to generate documentation? (a) // (b) / / (c) /* /" (Answer: c)

Lesson 2.6: ***Keywords and Naming Conventions***

- **Difficulty Level:** Beginner
- **Learning Outcome:** Recognize Java reserved keywords and apply standard naming conventions. (See also Lesson 1.6 for an introduction.)
- **Topics Covered:**
- Common keywords (`if`, `for`, `class`, `static`, etc.) and their purposes【6†L625-L632】
- Naming rules (no spaces, must start with letter/underscore, case sensitivity)
- Java naming conventions recap (from Lesson 1.6)
- **Practical Exercises:**
- Go through a list of words and identify which are Java keywords.
- Rename some poorly-named variables to follow proper conventions.
- **Mini Assignment:** Refactor code from previous lesson to correct any variable/method naming convention violations.
- **Common Mistakes:** Using keywords (like `class`, `static`) as identifiers; inconsistency in naming style.
- **Interview Questions:** "Name three keywords in Java that are not used anymore (deprecated keywords)." (Answer: `goto`, `const`, etc.)
- **Quiz Questions:** "Is '_myVar' a valid Java identifier? (Answer: Yes, but unconventional)"

Chapter 3: Control Flow Statements

- **Learning Objective:** Learn to control program execution using decision-making (if/else, switch) and looping constructs (for, while, do-while).
- **Skills Covered:** Writing conditional branches, switch-case usage, loop structures, and control flow adjustments using `break` and `continue`.
- **Prerequisites:** Chapter 2 (Java Basics).
- **Estimated Learning Time:** 5–7 hours.
- **Number of Lessons:** 4

Lesson 3.1: ***If-Else Statements***

- **Difficulty Level:** Beginner
- **Learning Outcome:** Implement conditional logic using `if`, `if-else`, and nested `if`.
- **Topics Covered:**
- `if` statement syntax, boolean expressions
- `if-else` and `else if` chains
- Nested `if` blocks
- Common pitfalls (missing braces)
- **Practical Exercises:**
- Write a program to determine if a number is positive, negative, or zero.
- Extend the program to check even/odd for positive numbers.
- **Mini Assignment:** Develop a simple grade classification system (A/B/C/D/F) based on numerical score using if-else.
- **Common Mistakes:** Omitting braces leading to logic errors; using assignment (=) instead of equality (==) in conditions.
- **Interview Questions:** "Explain the difference between `if-else` and `switch`."
- **Quiz Questions:** "Which operator is used to compare two values? (a) = (b) == (c) =!" (Answer: b)

Lesson 3.2: ***Switch Statement***

- **Difficulty Level:** Beginner
- **Learning Outcome:** Use the `switch` statement to make multi-way decisions based on an expression's value.
- **Topics Covered:**
- `switch` syntax with `case` labels and `default`
- `break` in switch to prevent fall-through
- Switching on `char`, `int`, `String` (Java 7+), and enum types
- **Practical Exercises:**
- Create a switch that prints the day of the week given an integer 1–7.

- Use switch to process simple menu options input by the user.
- **Mini Assignment:** Implement a basic calculator that takes an operator character (`+`, `-`, `*`, `/`) and two numbers, using switch to select the operation.
- **Common Mistakes:** Forgetting `break` causes fall-through; writing `switch` with only `if-else` logic.
- **Interview Questions:** "Can a `switch` statement use strings? Since which version?" (Answer: Yes, since Java 7.)
- **Quiz Questions:** "What happens if you omit the `break` in a switch case? (Answer: Execution continues into the next case.)"

Lesson 3.3: ***For and While Loops***

- **Difficulty Level:** Beginner
- **Learning Outcome:** Implement loops (`for`, `while`, `do-while`) for repeated execution.
- **Topics Covered:**
- `for` loop syntax (initialization; condition; update)
- `while` loop and `do-while` loop
- Loop examples: summation, factorial, etc.
- Off-by-one errors and loop termination.
- **Practical Exercises:**
- Print all even numbers from 1 to 20 using a `for` loop.
- Use a `while` loop to compute the sum of digits of an input number.
- **Mini Assignment:** Generate the Fibonacci sequence up to `n` terms using loops.
- **Common Mistakes:** Infinite loops due to missing update; misplacing semicolons after loop statements.
- **Interview Questions:** "When would you use a `do-while` loop instead of `while`?"
- **Quiz Questions:** "What is the output of this loop? `for(int i=0; i<3; i++) System.out.println(i);`" (Answer: 0 1 2)

Lesson 3.4: ***Nested Loops, Break, and Continue***

- **Difficulty Level:** Beginner
- **Learning Outcome:** Use nested loops for multi-dimensional iteration and control loop flow with `break` and `continue`.
- **Topics Covered:**
- Nested loops (e.g., printing patterns, 2D array traversal)
- `break` to exit loops, `continue` to skip to next iteration
- Labels (advanced use of break/continue with labels) – brief mention.
- **Practical Exercises:**
- Print a multiplication table using nested loops.
- Use nested loops to print a simple pattern (e.g., triangle of asterisks).

- **Mini Assignment:** Given a number grid (2D array), use nested loops to find the maximum value. Use `break` to exit early when a condition is met (e.g., value found).
- **Common Mistakes:** Breaking out of only inner loop when intending to exit outer loop; using `break` instead of `return` in some cases.
- **Interview Questions:** "What does the `continue` statement do in a loop?"
- **Quiz Questions:** "In nested loops, which `break` exits only the inner loop? (Answer: An unlabeled `break`.)"

Chapter 4: Methods (Functions)

- **Learning Objective:** Learn to encapsulate code into methods/functions, understanding parameters, return values, and method overloading.
- **Skills Covered:** Defining and invoking methods, using parameters and return types, overloading methods, and basic debugging of methods.
- **Prerequisites:** Chapters 1–3 (basic syntax and control flow).
- **Estimated Learning Time:** 4–6 hours.
- **Number of Lessons:** 4

Lesson 4.1: ***Defining and Calling Methods***

- **Difficulty Level:** Beginner
- **Learning Outcome:** Create methods with different return types and call them. Distinguish between static methods and instance methods.
- **Topics Covered:**
- Method syntax (return type, name, parameters)
- `void` methods vs methods returning values
- `static` keyword (static methods vs instance methods)
- Calling methods and passing arguments
- **Practical Exercises:**
- Write a method `sayHello()` that prints a greeting.
- Write a method `add(int a, int b)` that returns the sum of two integers. Call it from `main`.
- **Mini Assignment:** Build a `MathUtils` class with methods: `max(int, int)`, `min(int, int)`, and use them in a program.
- **Common Mistakes:** Forgetting to call a method; mismatching return type; not using `static` when calling from `static main`.
- **Interview Questions:** "What is a method signature in Java?"
- **Quiz Questions:** "True or False: In Java, methods can exist outside of classes. (Answer: False)"

Lesson 4.2: ***Method Parameters and Return Values***

- **Difficulty Level:** Beginner

- **Learning Outcome:** Use parameters (passed by value) in methods and understand return statements.
- **Topics Covered:**
- Passing parameters by value (primitive vs object reference)
- Returning values from methods, `return` statement, `void` vs non-void
- Scope of parameters (local scope)
- **Practical Exercises:**
- Write a method that takes a string and returns its length.
- Write a method that calculates the factorial of a number and returns it.
- **Mini Assignment:** Implement a method `calculateAverage(int[] nums)` that returns the average of an array of integers.
- **Common Mistakes:** Not returning a value from non-void method; confusing passing references vs values.
- **Interview Questions:** "Does Java support pass-by-reference? Explain." (Answer: No, it's always pass-by-value; object references are values.)
- **Quiz Questions:** "What will this code print? `System.out.println(add(2, 3));` given `int add(int x, int y){ return x+y; }`" (Answer: 5)

Lesson 4.3: ***Method Overloading***

- **Difficulty Level:** Intermediate
- **Learning Outcome:** Create multiple methods with the same name but different parameter lists (overloading) and know how Java resolves calls.
- **Topics Covered:**
- Overloading rules (different number/type/order of parameters)
- Difference between overloading and overriding
- Use cases for overloading (e.g., `print` methods, constructors)
- **Practical Exercises:**
- Overload a method `multiply` to handle both two `int` parameters and three `int` parameters.
- Overload a constructor in a class (e.g., `Rectangle(int width, int height)` and `Rectangle(int side)` for square).
- **Mini Assignment:** Create a `Calculator` class with overloaded methods: `compute(int, int)`, `compute(double, double)`, and test which is called.
- **Common Mistakes:** Changing only return type for overload (invalid); confusion when calling overloaded methods with similar arguments (type promotion).
- **Interview Questions:** "What is method overloading and when is it used?"
- **Quiz Questions:** "True or False: Overloaded methods must have different return types." (Answer: False)

Lesson 4.4: ***Recursion (Optional Intro)***

- **Difficulty Level:** Intermediate

- **Learning Outcome:** Understand the concept of recursion and when a method calls itself. Write a simple recursive method.
- **Topics Covered:**
- Recursive method structure (base case, recursive case)
- Example: factorial or Fibonacci using recursion
- Drawbacks of recursion (stack depth)
- **Practical Exercises:**
- Implement a recursive factorial function and test it.
- (Optional) Implement recursive Fibonacci and compare with iterative version.
- **Mini Assignment:** Write a recursive method to compute the greatest common divisor (GCD) of two numbers.
- **Common Mistakes:** Missing base case causing infinite recursion (StackOverflowError); inefficient recursion for large inputs.
- **Interview Questions:** "What is recursion? Can you give an example scenario where it is useful?"
- **Quiz Questions:** "What is the base case in recursion, and why is it important?" (Answer: The condition to stop recursion, to prevent infinite calls.)

Chapter 5: Arrays

- **Learning Objective:** Learn to store and manipulate collections of data using arrays.
- **Skills Covered:** Declaring and initializing 1D and 2D arrays, iterating through arrays, basic array algorithms (e.g., finding max/min, sorting).
- **Prerequisites:** Chapters 1–4.
- **Estimated Learning Time:** 4–6 hours.
- **Number of Lessons:** 3

Lesson 5.1: ***One-Dimensional Arrays***

- **Difficulty Level:** Beginner
- **Learning Outcome:** Declare, initialize, and manipulate one-dimensional arrays.
- **Topics Covered:**
- Array declaration (`int[] arr;`), creation (`new int[5]`), and initialization (inline and loops).
- Accessing elements by index, default values.
- Iterating with `for` and `for-each` loops.
- **Practical Exercises:**
- Create an array of 10 integers and fill it with values 1–10 using a loop.
- Calculate the sum and average of an array of numbers.

- **Mini Assignment:** Write a program to find the maximum and minimum element in a user-input array.
- **Common Mistakes:** ArrayIndexOutOfBounds by using wrong indices; confusion between `arr.length` and last index (`length-1`).
- **Interview Questions:** "What is the default value of array elements for int, boolean, and object arrays?" (Answer: 0, false, null).
- **Quiz Questions:** "How do you find the length of an array `arr`? (Answer: `arr.length`)"

Lesson 5.2: ***Multi-Dimensional Arrays (2D Arrays)***

- **Difficulty Level:** Beginner
- **Learning Outcome:** Work with 2D arrays (arrays of arrays) for grid-like data.
- **Topics Covered:**
- Declaring and creating 2D arrays (`int[][] matrix = new int[3][4];`)
- Accessing elements using two indices (`matrix[0][1]`)
- Iterating with nested loops
- **Practical Exercises:**
- Create a 3x3 matrix and fill it with multiplication table values (i*j).
- Compute the sum of each row or column in a 2D array.
- **Mini Assignment:** Given a 2D array (e.g., 5x5), find the diagonal sum (elements where row == column).
- **Common Mistakes:** Treating 2D arrays as single array; assuming `length` gives both dimensions (need `arr.length` and `arr[0].length`).
- **Interview Questions:** "How are 2D arrays stored in Java memory? (Answer: As an array of arrays.)"
- **Quiz Questions:** "What is the default value for elements in a 2D array of objects? (Answer: null)"

Lesson 5.3: ***Common Array Operations***

- **Difficulty Level:** Beginner
- **Learning Outcome:** Perform basic operations on arrays: searching, copying, sorting (intro).
- **Topics Covered:**
- Linear search in an array
- Using `Arrays` utility class (e.g., `Arrays.sort(arr)`, `Arrays.toString(arr)`)
- Shallow vs deep copy of arrays (`arr.clone()` vs manual)
- **Practical Exercises:**
- Use `Arrays.sort` to sort an array of random integers.
- Implement your own method to search for an element's index in an array.
- **Mini Assignment:** Merge two sorted arrays into a third sorted array.

- **Common Mistakes:** Sorting object array without implementing `Comparable`; copying arrays incorrectly leading to shared reference.
- **Interview Questions:** "How do you sort an array in Java?"
- **Quiz Questions:** "What does `Arrays.toString(array)` do? (Answer: Returns a string representation of the array's elements.)"

Chapter 6: Strings and StringBuilder

- **Learning Objective:** Master working with `String` objects and mutable `StringBuilder` for text processing.
- **Skills Covered:** Creating and manipulating strings, using common `String` methods, understanding immutability, and using `StringBuilder` for efficient modifications.
- **Prerequisites:** Chapters 1–5.
- **Estimated Learning Time:** 4–6 hours.
- **Number of Lessons:** 3

Lesson 6.1: ***String Basics and Immutability***

- **Difficulty Level:** Beginner
- **Learning Outcome:** Create and use `String` objects; understand that strings are immutable.
- **Topics Covered:**
- String literal vs `new String(...)`
- Immutability concept: concatenation creates new string, etc.
- `String` pool and `==` vs `.equals()` for comparison
- **Practical Exercises:**
- Create two strings with the same content using different methods and compare with `==` and `equals()`.
- Perform several concatenations and observe object creation (e.g., using `+`).
- **Mini Assignment:** Explain with code comments why strings are immutable and how the string pool works.
- **Common Mistakes:** Using `==` to compare string content; modifying a string in place (which is not possible).
- **Interview Questions:** "Why are strings immutable in Java?"
- **Quiz Questions:** "Which method compares the contents of two strings? (Answer: `equals()`)"

Lesson 6.2: ***String Methods***

- **Difficulty Level:** Beginner
- **Learning Outcome:** Use common `String` methods for manipulation and queries.
- **Topics Covered:**

- Length, `charAt`, `substring`, `contains`, `startsWith`, `endsWith`
- Case conversion (`toUpperCase()`, `toLowerCase()`), `trim()`, `replace()`, `split()`
- Converting numbers to strings (`String.valueOf()`) and vice versa (`Integer.parseInt()`)
- **Practical Exercises:**
- Write a program that checks if a sentence contains the word "Java" (case-insensitive).
- Parse a CSV string using `split` and print each value.
- **Mini Assignment:** Build a simple palindrome checker that ignores case and spaces by using string methods.
- **Common Mistakes:** Off-by-one in `substring` indices; Not handling `NullPointerException` on null strings.
- **Interview Questions:** "How to compare two strings ignoring case?" (Answer: `equalsIgnoreCase()`)
- **Quiz Questions:** "What does `str.substring(2, 5)` return from `"HelloWorld"`? (Answer: `"llo"` if 0-based index)"

Lesson 6.3: ***StringBuilder and StringBuffer***

- **Difficulty Level:** Intermediate
- **Learning Outcome:** Use `StringBuilder` (or `StringBuffer`) for efficient string concatenation and modification.
- **Topics Covered:**
- Difference between `String` and `StringBuilder` (mutable sequence of characters)
- Key methods: `append()`, `insert()`, `reverse()`, `toString()`
- When to use `StringBuilder` (e.g., in loops)
- **Practical Exercises:**
- Concatenate numbers 1–1000 into a single string first using + (measure time) and then using `StringBuilder.append()` (compare performance).
- Use `StringBuilder` to reverse a user-input string.
- **Mini Assignment:** Create a `StringBuilder` and use it to build a comma-separated list of user-entered words.
- **Common Mistakes:** Forgetting to call `toString()` when outputting a `StringBuilder`; using `StringBuffer` unnecessarily (thread-safe but slower).
- **Interview Questions:** "Why might `StringBuilder` be preferred over `String` in loops?"
- **Quiz Questions:** "Does `StringBuilder` override `toString()`? (Answer: Yes, to return a `String`.)"

Chapter 7: Object-Oriented Programming (OOP) Basics

- **Learning Objective:** Grasp fundamental OOP concepts: defining classes/objects, constructors, the `this` keyword, and static/final modifiers. Apply encapsulation (access modifiers).
- **Skills Covered:** Designing classes, instantiating objects, constructor use, using `static` variables/methods and constants with `final`, and encapsulation with private fields and public getters/setters.
- **Prerequisites:** Chapters 1–6.
- **Estimated Learning Time:** 6–8 hours.
- **Number of Lessons:** 4

Lesson 7.1: ***Classes and Objects***

- **Difficulty Level:** Beginner
- **Learning Outcome:** Create custom classes and objects, and understand the relationship between them.
- **Topics Covered:**
- Defining a class (fields and methods), creating object instances with `new`
- Instance variables vs class variables (static) overview
- Using default constructor vs custom constructors
- **Practical Exercises:**
- Define a `Person` class with `name` and age fields; create and print two `Person` objects.
- Modify the `Person` class to add a method `printInfo()` that displays details.
- **Mini Assignment:** Create a `Book` class with fields (title, author, price) and methods to display details; then instantiate objects and use them.
- **Common Mistakes:** Forgetting to create objects (`NullPointerException`); treating classes like objects without instantiation.
- **Interview Questions:** "What is the difference between a class and an object in Java?"
- **Quiz Questions:** "Which keyword is used to create a new object? (Answer: new)"

Lesson 7.2: ***Constructors and `this` Keyword***

- **Difficulty Level:** Beginner
- **Learning Outcome:** Use constructors to initialize objects and use `this` to distinguish instance variables.
- **Topics Covered:**
- Constructor declaration (`ClassName(...)`), default constructor vs parameterized constructor
- Overloading constructors (multiple constructors)
- Using `this` to refer to current object's fields/other constructors (`this(...)`)
- **Practical Exercises:**

- Add a parameterized constructor to the Book class to set title and author.
- Use `this` in a constructor to call another constructor in the same class (constructor chaining).
- **Mini Assignment:** Create a `Rectangle` class with width and height, providing two constructors: one default (1x1) and one taking width/height.
- **Common Mistakes:** Forgetting to initialize all fields; shadowing variables (same name for parameter and field) without using `this`.
- **Interview Questions:** "Can a constructor return a value? (Answer: No, constructors do not have a return type.)"
- **Quiz Questions:** "What does the `this` keyword represent in Java? (Answer: the current object)"

Lesson 7.3: ***Static and Final***

- **Difficulty Level:** Intermediate
- **Learning Outcome:** Understand `static` members (shared among all instances) and `final` keyword (constants and immutability aspects).
- **Topics Covered:**
- `static` fields and methods (class-level vs instance-level)
- Accessing static methods (`ClassName.method()`) vs instance methods
- `final` variables (constants) and `final` classes/methods (cannot override/extend)
- **Practical Exercises:**
- Add a `static` counter to Book class to count the number of books created.
- Define a constant `MAX_SIZE` in a class with `final` and try to modify it (compiler error).
- **Mini Assignment:** Create a `MathConstants` class with public static final fields (e.g., `PI`, `E`) and use them in calculations.
- **Common Mistakes:** Trying to use instance variable in a static method; forgetting to qualify static fields.
- **Interview Questions:** "What is a static method? Give an example from Java API." (Answer: e.g., `Math.abs()` is static.)
- **Quiz Questions:** "Can a final variable be changed once initialized? (Answer: No)"

Lesson 7.4: ***Encapsulation and Access Modifiers***

- **Difficulty Level:** Beginner
- **Learning Outcome:** Apply encapsulation by using access modifiers (`private`, `public`, `protected`) and provide controlled access to class fields.
- **Topics Covered:**
- Access modifiers: `private`, `public`, `protected`, (package-private)
- Getters and setters for private fields
- Benefits of encapsulation (hiding internal state)
- **Practical Exercises:**

- Make fields of `Book` class private and generate public getter/setter methods.
- Try accessing a private field from another class and observe the error.
- **Mini Assignment:** Implement a class `Account` with private balance and provide methods to deposit, withdraw (demonstrating encapsulation).
- **Common Mistakes:** Making everything `public`; no validation in setters leading to invalid states.
- **Interview Questions:** "Why is encapsulation important?"
- **Quiz Questions:** "Which keyword makes a member accessible only within its own class? (Answer: private)"

Chapter 8: Inheritance and Polymorphism

- **Learning Objective:** Extend classes to promote code reuse (inheritance) and use polymorphism (overriding, dynamic dispatch) for flexible designs.
- **Skills Covered:** Creating subclass/parent classes, using `extends`, method overriding, upcasting/downcasting, and understanding polymorphic behavior.
- **Prerequisites:** Chapter 7 (OOP basics).
- **Estimated Learning Time:** 5–7 hours.
- **Number of Lessons:** 3

Lesson 8.1: ***Inheritance (Extends and super)***

- **Difficulty Level:** Intermediate
- **Learning Outcome:** Create a subclass that inherits fields and methods from a superclass, using the `extends` keyword and calling superclass constructors/methods.
- **Topics Covered:**
- Using `extends` to create a child class
- Inheriting methods/fields vs private members
- `super()` to call parent constructor, `super` to access parent's fields/methods
- `@Override` annotation
- **Practical Exercises:**
- Create a base class `Animal` with a method `makeSound()`, then subclass `Dog extends Animal` and override `makeSound()`.
- Use `super` in constructor of `Dog` to initialize base class parts (e.g., name).
- **Mini Assignment:** Design a class hierarchy: `Vehicle` (fields: make, model) and subclasses `Car` and `Motorcycle`, each with additional fields. Demonstrate inheritance.
- **Common Mistakes:** Forgetting to call parent constructor (if no default constructor); using overriding incorrectly (signature mismatch).

- **Interview Questions:** "What is single inheritance? Does Java support multiple inheritance with classes?" (Answer: Only single inheritance for classes; multiple inheritance via interfaces.)
- **Quiz Questions:** "How do you call the parent class's constructor from a subclass? (Answer: using `super(...)`)"

Lesson 8.2: ***Polymorphism (Method Overriding and Dynamic Dispatch)***

- **Difficulty Level:** Intermediate
- **Learning Outcome:** Use polymorphism to call the appropriate overridden method at runtime and understand upcasting/downcasting.
- **Topics Covered:**
- Overriding methods (same signature in subclass)
- Polymorphic variables (e.g., `Animal a = new Dog()`)
- Dynamic dispatch at runtime chooses subclass method
- Casting objects: upcasting (safe), downcasting with `instanceof` check
- **Practical Exercises:**
- Create a list of `Animal` references containing different subclass objects, and call `makeSound()` on each to see polymorphism.
- Attempt a downcast: `Animal a = new Dog(); Dog d = (Dog) a;` and use `instanceof` to check type before casting.
- **Mini Assignment:** Implement a method that takes a parameter of type `Shape` (base class) and prints area; test it by passing a `Circle` or `Rectangle` object.
- **Common Mistakes:** ClassCastException when casting to wrong type; forgetting to mark method as `@Override` (not mandatory but good practice).
- **Interview Questions:** "What is polymorphism in Java? How is it achieved?"
- **Quiz Questions:** "Given `Shape s = new Circle(); s.draw();` which class's `draw()` is invoked? (Answer: Circle's)"

Lesson 8.3: ***Abstract Classes***

- **Difficulty Level:** Intermediate
- **Learning Outcome:** Define abstract classes and methods to enforce method implementation in subclasses.
- **Topics Covered:**
- `abstract` keyword for classes and methods
- Cannot instantiate abstract class directly
- Subclasses must override abstract methods or be abstract themselves
- Use cases for abstraction (common interface with partial implementation)
- **Practical Exercises:**
- Define an abstract class `Account` with abstract method `calculateInterest()`. Create `SavingsAccount` subclass that implements it.
- Try to instantiate `Account` (should cause error).

- **Mini Assignment:** Create an abstract class `Employee` with abstract method `calculatePay()`, and concrete subclasses for `HourlyEmployee` and `SalariedEmployee`.
- **Common Mistakes:** Not implementing all abstract methods in subclass; forgetting `abstract` keyword in method declaration.
- **Interview Questions:** "Can abstract classes have constructors?" (Answer: Yes, but they cannot be instantiated.)
- **Quiz Questions:** "True or False: Abstract methods can have a body. (Answer: False)"

Chapter 9: Interfaces and Packages

- **Learning Objective:** Learn to define and implement interfaces for multiple inheritance of behavior and organize classes into packages.
- **Skills Covered:** Creating interfaces, implementing them, using default/static interface methods (Java 8+), and organizing code into packages with proper access.
- **Prerequisites:** Chapter 8 (Inheritance/Polymorphism).
- **Estimated Learning Time:** 4–6 hours.
- **Number of Lessons:** 2

Lesson 9.1: ***Interfaces (Java 8+ including Default Methods)***

- **Difficulty Level:** Intermediate
- **Learning Outcome:** Define interfaces and implement them in classes. Understand interface benefits over abstract classes for multiple inheritance.
- **Topics Covered:**
- Declaring an interface (`interface MyInterface`) and implementing (`implements`)
- Interface methods are abstract by default (before Java 8)
- Java 8+ default and static methods in interfaces
- Multiple interfaces implemented by a class (multiple inheritance of type)
- **Practical Exercises:**
- Create an interface `Movable` with method `move()`. Implement it in classes `Robot` and `Animal`.
- Add a default method in `Movable` (e.g., `default void stop()`) and use it in implementing classes.
- **Mini Assignment:** Design an interface `Playable` for games with methods `play()` and a default `pause()`. Have two game classes implement it.
- **Common Mistakes:** Not using `@Override` for interface methods; trying to instantiate an interface.
- **Interview Questions:** "Can a Java class implement multiple interfaces? Give an example."

- **Quiz Questions:** "Which keyword is used by a class to implement an interface? (Answer: implements)"

Lesson 9.2: ***Packages and Access Control***

- **Difficulty Level:** Beginner
- **Learning Outcome:** Organize classes into packages and use import statements. Understand package-private (default) access.
- **Topics Covered:**
- `package` declaration at top of Java file
- Directory structure corresponding to packages
- Using `import` to include classes from other packages
- Access levels: public, protected, package-private, private (review)
- **Practical Exercises:**
- Create a package `com.example.utils` and place a utility class there. Import and use it from `com.example.main`.
- Experiment with omitting `import` and using fully qualified class names.
- **Mini Assignment:** Refactor a set of classes into two packages (e.g., `model` and `service`) and ensure everything compiles with the correct imports.
- **Common Mistakes:** Forgetting `package` line or mismatching folder structure; circular package dependencies.
- **Interview Questions:** "What is the default (package-private) access level in Java?"
- **Quiz Questions:** "If no access modifier is specified, which classes can access a member? (Answer: Classes in the same package)"

Chapter 10: Exception Handling

- **Learning Objective:** Learn to write robust programs by handling runtime errors using exceptions.
- **Skills Covered:** Using `try-catch-finally`, understanding checked vs unchecked exceptions, throwing custom exceptions.
- **Prerequisites:** Chapters 1–9.
- **Estimated Learning Time:** 4–5 hours.
- **Number of Lessons:** 3

Lesson 10.1: ***Exception Basics (try-catch-finally)***

- **Difficulty Level:** Intermediate
- **Learning Outcome:** Handle exceptions using `try`, `catch`, and `finally` blocks.
- **Topics Covered:**
- Checked exceptions vs unchecked exceptions (RuntimeException)
- Writing `try { ... } catch (ExceptionType e) { ... } finally { ... }`

- Multiple catch blocks and the order (subclass before superclass)
- **Practical Exercises:**
- Write code that divides two numbers and use try-catch to handle `ArithmeticException` (divide by zero).
- Demonstrate `finally` by opening/closing a resource (simulate with print statements).
- **Mini Assignment:** Create a method that reads an integer from user and catches `InputMismatchException` if invalid.
- **Common Mistakes:** Catching general `Exception` instead of specific ones; forgetting `finally` block.
- **Interview Questions:** "What is the difference between `throw` and `throws` in Java?"
- **Quiz Questions:** "Which block is always executed regardless of exceptions? (Answer: finally)"

Lesson 10.2: ***Common Exception Types and Throwing***

- **Difficulty Level:** Intermediate
- **Learning Outcome:** Recognize common Java exception types (e.g., `NullPointerException`, `ArrayIndexOutOfBoundsException`) and throw exceptions manually.
- **Topics Covered:**
- Examples: `NullPointerException`, `NumberFormatException`, `FileNotFoundException` (checked)
- Using `throw` to throw a new exception, and `throws` in method signature
- Custom exceptions: creating a class that extends `Exception` or `RuntimeException`
- **Practical Exercises:**
- Write a code snippet that causes a `NullPointerException` and catch it.
- Throw an `IllegalArgumentException` in a method if an argument is invalid.
- **Mini Assignment:** Define a custom exception `InvalidAgeException` thrown when age < 0 in a setter.
- **Common Mistakes:** Not declaring checked exceptions in method signature; throwing generic `Exception` instead of specific types.
- **Interview Questions:** "What happens if you don't catch a checked exception that a method throws?" (Answer: Compilation error unless method declares `throws`.)
- **Quiz Questions:** "True or False: `NullPointerException` is a checked exception. (Answer: False)"

Lesson 10.3: ***Try-With-Resources (Java 7+)***

- **Difficulty Level:** Intermediate
- **Learning Outcome:** Use try-with-resources for automatic resource management (e.g., auto-closing streams).
- **Topics Covered:**

- Syntax: `try (ResourceType res = new ResourceType()) { ... }`
- Resources must implement `AutoCloseable` (e.g., streams, JDBC connections)
- Benefits: no need for explicit `finally` close
- **Practical Exercises:**
- Read a file line-by-line using `try-with-resources` and automatically close the reader.
- Catch and handle possible IOExceptions in the process.
- **Mini Assignment:** Modify previous file I/O tasks to use try-with-resources.
- **Common Mistakes:** Forgetting `finally` with regular try; not closing streams leads to resource leaks.
- **Interview Questions:** "What interfaces must a resource implement to be used in try-with-resources? (Answer: AutoCloseable)"
- **Quiz Questions:** "Which Java version introduced try-with-resources? (Answer: Java 7)"

Chapter 11: File Handling and I/O Streams

- **Learning Objective:** Learn to read from and write to files using Java I/O APIs.
- **Skills Covered:** Using `File`, `FileReader`, `FileWriter`, buffered streams, and understanding file paths.
- **Prerequisites:** Chapter 10 (Exception Handling).
- **Estimated Learning Time:** 4–6 hours.
- **Number of Lessons:** 3

*Lesson 11.1: **Working with Files (File class)***

- **Difficulty Level:** Beginner
- **Learning Outcome:** Use `java.io.File` to handle file paths and basic file operations.
- **Topics Covered:**
- Creating `File` objects, absolute vs relative paths
- Checking file existence, isDirectory, createNewFile, delete, mkdir
- Listing files in a directory (`listFiles()`)
- **Practical Exercises:**
- Create a `File` object for a directory and print its contents.
- Check if a given file path exists; if not, create the file.
- **Mini Assignment:** Write a program that creates a directory "testDir" and inside it creates 3 empty text files named "file1.txt", "file2.txt", "file3.txt".
- **Common Mistakes:** Forgetting to handle `IOException`; confusing file paths (relative path vs working directory).

- **Interview Questions:** “How do you check if a file exists in Java?” (Answer: `file.exists()`)
- **Quiz Questions:** “Which class represents file and directory pathnames in Java? (Answer: File)”

Lesson 11.2: ***Reading Text Files***

- **Difficulty Level:** Beginner
- **Learning Outcome:** Read text files using `FileReader` and `BufferedReader`.
- **Topics Covered:**
- `FileReader` and wrapping it in `BufferedReader` for efficiency
- Reading lines with `readLine()` in a loop
- Handling `IOException`
- **Practical Exercises:**
- Read a file “input.txt” and print its contents line by line.
- Count the number of lines in a file using `BufferedReader`.
- **Mini Assignment:** Create a program that reads a CSV file and splits each line into fields.
- **Common Mistakes:** Not closing the stream (use try-with-resources to avoid this); reading until EOF incorrectly.
- **Interview Questions:** “What is the purpose of `BufferedReader` in file I/O?”
- **Quiz Questions:** “Which method reads a line of text from a `BufferedReader`? (Answer: readLine())”

Lesson 11.3: ***Writing Text Files***

- **Difficulty Level:** Beginner
- **Learning Outcome:** Write text data to files using `FileWriter` and `BufferedWriter`.
- **Topics Covered:**
- `FileWriter` and `BufferedWriter` (or `PrintWriter`)
- Writing strings and newlines to files (`write()`, `newLine()`, `printf()`
- Appending to existing files vs overwriting
- **Practical Exercises:**
- Write user input to a file named “output.txt” (e.g., the user’s diary entries).
- Append a timestamped log message to a log file.
- **Mini Assignment:** Generate a report file that lists numbers 1–100 and whether they are prime or not.
- **Common Mistakes:** Not closing writers leading to no data in file; writing without flushing.
- **Interview Questions:** “How do you write text to a file in Java?”
- **Quiz Questions:** “True or False: `FileWriter` alone automatically buffers output. (Answer: False, use BufferedWriter for buffering.)”

Chapter 12: Collections Framework – Lists

- **Learning Objective:** Learn to use the `List` interface and its implementations (`ArrayList`, `LinkedList`) for dynamic collections.
- **Skills Covered:** Creating and manipulating lists, adding/removing elements, iterating over lists, using enhanced for-loops, and basic list algorithms.
- **Prerequisites:** Chapter 4 (Arrays) and 7 (Basic OOP).
- **Estimated Learning Time:** 5–6 hours.
- **Number of Lessons:** 3

Lesson 12.1: ***List and ArrayList***

- **Difficulty Level:** Beginner
- **Learning Outcome:** Use `ArrayList` to dynamically store ordered data, and perform basic list operations.
- **Topics Covered:**
- `List` interface overview, `ArrayList` instantiation (`new ArrayList<>()`)
- Adding (`add`), accessing (`get`), removing (`remove`), and modifying (`set`) elements
- Iterating with `for`, `for-each`, and `Iterator`
- **Practical Exercises:**
- Create an `ArrayList` of strings, add several names, sort them (using `Collections.sort`), and print.
- Remove duplicate elements from an `ArrayList` of integers by using a secondary list.
- **Mini Assignment:** Build a to-do list application storing tasks in an `ArrayList` and allowing adding/removing tasks.
- **Common Mistakes:** Using == to compare list contents; not specifying generic type leads to raw type warnings.
- **Interview Questions:** "When should you use an `ArrayList` over an array?"
- **Quiz Questions:** "How do you add an element x to an `ArrayList<Integer> list`? (Answer: list.add(x))"

Lesson 12.2: ***LinkedList***

- **Difficulty Level:** Beginner
- **Learning Outcome:** Use `LinkedList` for efficient insertions/deletions and understand differences from `ArrayList`.
- **Topics Covered:**
- Creating `LinkedList`, using it as `List` or as `Queue`
- Performance: O(1) insertions at ends vs `ArrayList` O(n) for mid-list insertion
- Using `addFirst`, `addLast`, `removeFirst`, `removeLast`
- **Practical Exercises:**

- Insert elements at the beginning and end of a `LinkedList` and compare operations.
- Use `LinkedList` to implement a simple queue (offer, poll).
- **Mini Assignment:** Convert the to-do list from previous lesson to use a `LinkedList` and note any differences.
- **Common Mistakes:** Expecting random access (`get(index)`) to be fast on `LinkedList` (it's O(n)).
- **Interview Questions:** "What is the time complexity of accessing an element by index in a LinkedList?" (Answer: O(n)).
- **Quiz Questions:** "Which List implementation would you use for frequent random access? (Answer: ArrayList)"

Lesson 12.3: ***List Algorithms and Utilities***

- **Difficulty Level:** Intermediate
- **Learning Outcome:** Apply common algorithms on lists and use utility methods.
- **Topics Covered:**
- `Collections` utility class: `sort`, `reverse`, `shuffle`, `binarySearch`
- Converting arrays to lists (`Arrays.asList`) and vice versa
- `List` iteration with `Iterator` and `ListIterator`
- **Practical Exercises:**
- Given a sorted list of integers, use `Collections.binarySearch` to find elements.
- Shuffle a list of strings and print the new order.
- **Mini Assignment:** Write a function that takes a list and returns a new list with elements in reverse order without using `Collections.reverse`.
- **Common Mistakes:** Modifying list while iterating without using Iterator's `remove`; forgetting to sort before using `binarySearch`.
- **Interview Questions:** "What happens if you call `binarySearch` on an unsorted list?" (Answer: result is undefined).
- **Quiz Questions:** "Which class provides static methods like sort and shuffle for lists? (Answer: Collections)"

Chapter 13: Collections – Sets and Queues

- **Learning Objective:** Understand collection types for unique elements (`Set`) and queues (`Queue`, `Deque`), and choose appropriate implementations (`HashSet`, `TreeSet`, `PriorityQueue`, etc.).
- **Skills Covered:** Using `Set` interface (`HashSet`, `TreeSet`), `Queue` interface (`PriorityQueue`, `Deque`), handling unordered and sorted collections.
- **Prerequisites:** Chapter 12 (Lists).
- **Estimated Learning Time:** 5–7 hours.
- **Number of Lessons:** 3

Lesson 13.1: ***Set Interface (HashSet, TreeSet)***

- **Difficulty Level:** Intermediate
- **Learning Outcome:** Use `Set` collections to store unique elements; differentiate between `HashSet` (unordered) and `TreeSet` (sorted).
- **Topics Covered:**
- `Set` interface, no duplicates property
- `HashSet`: quick insertion and lookup (backed by hash table)
- `TreeSet`: sorted order, implements `NavigableSet`
- Iterating over sets (order differences)
- **Practical Exercises:**
- Add duplicate elements to a `HashSet` and observe that duplicates are ignored.
- Use a `TreeSet` to sort a list of strings automatically.
- **Mini Assignment:** Given a list of words, create a set to find the number of unique words.
- **Common Mistakes:** Assuming insertion order is preserved in `HashSet`; forgetting that `TreeSet` requires elements to be `Comparable` or use a `Comparator`.
- **Interview Questions:** "When would you choose a `HashSet` over a `TreeSet`?"
- **Quiz Questions:** "What is the primary difference between HashSet and TreeSet? (Answer: ordering)"

Lesson 13.2: ***Queue Interface (PriorityQueue, Deque)***

- **Difficulty Level:** Intermediate
- **Learning Outcome:** Use `Queue` and `Deque` for first-in-first-out (FIFO) operations and understand priority queues.
- **Topics Covered:**
- `Queue` interface, methods `offer`, `poll`, `peek`
- `LinkedList` as a queue (using `offer`/`poll`)
- `PriorityQueue`: natural ordering or custom comparator for priority
- `Deque` interface with `ArrayDeque` (double-ended queue operations `addFirst`/`addLast`)
- **Practical Exercises:**
- Simulate a line of customers using a `LinkedList` as a `Queue`.
- Use `PriorityQueue` to process tasks with priorities (e.g., smallest number first).
- **Mini Assignment:** Implement a simple print job scheduler where lower job numbers have higher priority using `PriorityQueue`.
- **Common Mistakes:** Using `remove()` vs `poll()` (throwing exception vs null return); not providing comparator leads to ClassCastException.
- **Interview Questions:** "What happens if you poll an empty `PriorityQueue`?" (Answer: returns null).

- **Quiz Questions:** "How do you insert an element into a Java `Queue`? (Answer: `offer(element)`)"

Lesson 13.3: ***Deque (Double-Ended Queue)***

- **Difficulty Level:** Intermediate
- **Learning Outcome:** Utilize `Deque` for stack and queue operations with a double-ended queue.
- **Topics Covered:**
- `Deque` interface: `ArrayDeque` implementation
- Using as stack: `push`, `pop`, `peek`
- Using as queue: `offerFirst`, `offerLast`, `pollFirst`, `pollLast`
- **Practical Exercises:**
- Use `ArrayDeque` to reverse a list by pushing all elements then popping them.
- Implement a sliding window algorithm that uses deque to track maximum of subarrays.
- **Mini Assignment:** Create a browser history simulation using `Deque` (forward/back navigation using push/pop).
- **Common Mistakes:** Using `null` with `ArrayDeque` (it doesn't allow null elements); confusion between methods returning vs throwing on empty deque.
- **Interview Questions:** "Why might `ArrayDeque` be preferred over `Stack` for stack implementation?"
- **Quiz Questions:** "Which method adds an element at the front of a deque? (Answer: addFirst or offerFirst)"

Chapter 14: Collections – Maps, Iterators, Comparable, Comparator

- **Learning Objective:** Use `Map` collections for key-value data, and understand object comparison mechanisms (`Comparable`, `Comparator`, `Iterator`).
- **Skills Covered:** Using `HashMap`, `TreeMap`, `Iterator`, and implementing comparison for sorting/custom maps.
- **Prerequisites:** Chapter 13 (Sets/Queues).
- **Estimated Learning Time:** 5–7 hours.
- **Number of Lessons:** 4

Lesson 14.1: ***Map Interface (HashMap)***

- **Difficulty Level:** Intermediate
- **Learning Outcome:** Store and retrieve data via key-value pairs using `HashMap`.
- **Topics Covered:**
- `Map` interface, `HashMap` implementation (unsorted, allows null key/value)
- Basic operations: `put`, `get`, `remove`, `containsKey`, `containsValue`
- Iterating over keys, values, entries (using `keySet()`, `values()`, `entrySet()`)

- **Practical Exercises:**
- Create a phone book using `HashMap<String, String>` mapping names to numbers. Implement lookup and removal.
- Count frequency of words in a text by using a `HashMap<String, Integer>`.
- **Mini Assignment:** Develop a simple student grade lookup using a map of student IDs to grades, with add/remove operations.
- **Common Mistakes:** Expecting map keys to be ordered; modifying map while iterating without using iterator.
- **Interview Questions:** "What is the complexity of `HashMap.get(key)` on average? (Answer: O(1))"
- **Quiz Questions:** "How do you retrieve a value from a HashMap with key k? (Answer: map.get(k))"

Lesson 14.2: ***TreeMap and Sorted Maps***

- **Difficulty Level:** Intermediate
- **Learning Outcome:** Use `TreeMap` to maintain a sorted map of keys.
- **Topics Covered:**
- `TreeMap` (keys in natural order or via custom `Comparator`)
- Difference: `HashMap` (unsorted) vs `TreeMap` (sorted by keys)
- `Map.Entry` and `firstKey()`, `lastKey()`, `subMap()` operations in sorted maps
- **Practical Exercises:**
- Create a `TreeMap<Integer, String>` of student scores to names; print entries in ascending order of scores.
- Use `subMap()` to get a range of keys.
- **Mini Assignment:** Given sales data in a `TreeMap<Date, Integer>`, find all entries between two dates.
- **Common Mistakes:** Using mutable objects as keys without proper comparator; forgetting keys must be comparable.
- **Interview Questions:** "How does a TreeMap order its keys by default?" (Answer: natural order of keys' Comparable implementation)
- **Quiz Questions:** "Which map guarantees sorted order of keys? (Answer: TreeMap)"

Lesson 14.3: ***Iterator and Iterable***

- **Difficulty Level:** Beginner
- **Learning Outcome:** Use `Iterator` and enhanced for-loop to traverse collections uniformly.
- **Topics Covered:**
- `Iterable` interface, using `for-each` loop for any iterable (List, Set, etc.)
- `Iterator` interface: `hasNext()`, `next()`, and safe `remove()` during iteration
- Fail-fast behavior of Java collections (ConcurrentModificationException)

- **Practical Exercises:**
- Traverse a list and remove certain elements while iterating (using `iterator.remove()`).
- Compare using a for-loop vs iterator for a collection.
- **Mini Assignment:** Implement a custom class that implements `Iterable` to iterate over an internal array.
- **Common Mistakes:** Removing elements from a list in a for-each loop (causes exceptions); not checking `hasNext()`.
- **Interview Questions:** "What exception is thrown when a collection is modified during iteration?" (Answer: ConcurrentModificationException)
- **Quiz Questions:** "How do you manually remove the current element from a collection using an iterator? (Answer: iterator.remove())"

Lesson 14.4: ***Comparable and Comparator***

- **Difficulty Level:** Intermediate
- **Learning Outcome:** Implement `Comparable` in classes for natural ordering, and use `Comparator` for custom ordering.
- **Topics Covered:**
- `Comparable<T>` interface and `compareTo()` method for natural sorting
- `Comparator<T>` interface (lambda or anonymous class) for custom sorting
- Using `Collections.sort(list)` for objects (requires Comparable) or `Collections.sort(list, comparator)`
- **Practical Exercises:**
- Create a `Person` class with age and name. Implement `Comparable<Person>` to sort by age.
- Use a `Comparator<Person>` to sort by name instead (alphabetically).
- **Mini Assignment:** Given a list of `Employee` objects (fields `id`, `salary`), sort them by salary using `Comparator.comparing()`.
- **Common Mistakes:** Returning incorrect values from `compareTo` (e.g., subtracting without checking overflow); forgetting to adhere to contract (transitive).
- **Interview Questions:** "What's the difference between `Comparable` and `Comparator` in Java?"
- **Quiz Questions:** "If a class implements `Comparable`, which method must it override? (Answer: compareTo())"

Chapter 15: Generics

- **Learning Objective:** Use Java Generics to write type-safe, reusable classes and methods (e.g., generic collections).
- **Skills Covered:** Defining generic classes and methods, using bounded type parameters, and understanding type erasure.

- **Prerequisites:** Chapter 14 (Collections) and Chapter 4 (Methods).
- **Estimated Learning Time:** 3–4 hours.
- **Number of Lessons:** 2

Lesson 15.1: ***Generic Classes and Methods***

- **Difficulty Level:** Intermediate
- **Learning Outcome:** Create and use generic classes (e.g., `Box<T>`) and methods for type flexibility.
- **Topics Covered:**
- Generic class syntax `<T>` (e.g., `class Box<T> { ... }`)
- Generic methods (e.g., `<E> void printArray(E[] arr)`)
- Type inference in method calls (`Collections.<String>emptyList()` implicitly)
- **Practical Exercises:**
- Implement a generic class `Pair<T, U>` to hold two related objects of different types.
- Write a generic method `swap(T[] arr, int i, int j)` to swap elements in an array.
- **Mini Assignment:** Create a generic `Container<T>` that stores an object and can return it; demonstrate with different types (`Integer`, `String`).
- **Common Mistakes:** Mixing raw types and generic types (causes warnings); using primitives as type parameters (use wrapper types instead).
- **Interview Questions:** "Why does Java use type erasure for generics?"
- **Quiz Questions:** "Can you create a generic array `T[] arr = new T[10]`? Why or why not? (Answer: No, due to type erasure)"

Lesson 15.2: ***Bounded Types and Wildcards***

- **Difficulty Level:** Intermediate
- **Learning Outcome:** Use bounded type parameters (`<T extends ...>`) and wildcards (`? extends`, `? super`) in generics.
- **Topics Covered:**
- Bounded type parameters (e.g., `class Box<T extends Number>`)
- Wildcards in generic references (PECS rule: Producer Extends, Consumer Super)
- Examples: `List<? extends Number>` vs `List<? super Integer>`
- **Practical Exercises:**
- Write a method `printList(List<? extends Number> list)` that prints elements of a number list.
- Demonstrate adding to `List<? super Integer>` vs `List<? extends Integer>`.
- **Mini Assignment:** Write a generic method `sum(List<? extends Number> list)` that returns the sum of numbers.

- **Common Mistakes:** Attempting to add elements to a `List<? extends T>`; misunderstanding wildcard bounds.
- **Interview Questions:** "Explain PECS mnemonic in Java generics." (Answer: Producer extends, Consumer super)
- **Quiz Questions:** "Which wildcard allows writing to a list? `? extends Number` or `? super Number`? (Answer: ? super Number)"

Chapter 16: Enums and Annotations (Basics)

- **Learning Objective:** Use Java `enum` types to define a fixed set of constants, and understand basic annotations.
- **Skills Covered:** Declaring and using enums, adding fields/methods to enums, and using built-in annotations like `@Override` and `@Deprecated`.
- **Prerequisites:** Chapters 1–15.
- **Estimated Learning Time:** 3–4 hours.
- **Number of Lessons:** 2

Lesson 16.1: ***Enums***

- **Difficulty Level:** Beginner
- **Learning Outcome:** Declare and use `enum` types with possible additional fields and methods.
- **Topics Covered:**
- Simple `enum` declaration (e.g., `enum Day { MONDAY, TUESDAY, ... }`)
- `enum` in switch statements (Java 7+), `values()` and `valueOf()` methods
- Adding fields, constructors, and methods to enums
- **Practical Exercises:**
- Create an `enum` for the four seasons with average temperature fields, and print each season's info.
- Use an enum in a `switch` to determine if a day is a weekday or weekend.
- **Mini Assignment:** Implement an `enum` for `Direction` (NORTH, EAST, SOUTH, WEST) with a method to get the opposite direction.
- **Common Mistakes:** Forgetting semicolon when adding fields/methods to enum; not using uppercase for enum constants (convention).
- **Interview Questions:** "How do you iterate through all values of an enum? (Answer: Using the static `values()` method.)"
- **Quiz Questions:** "Can enums have constructors in Java? (Answer: Yes, private constructors.)"

Lesson 16.2: ***Annotations (Basic Use)***

- **Difficulty Level:** Beginner

- **Learning Outcome:** Understand and apply common Java annotations to improve code clarity and error checking.
- **Topics Covered:**
- Purpose of annotations (`@Override`, `@Deprecated`, `@SuppressWarnings`)
- Using `@Override` to ensure correct method overriding
- (Brief mention) Defining a custom annotation (advanced, optional)
- **Practical Exercises:**
- Annotate overridden methods with `@Override`. Intentionally cause a mismatch to see compile error.
- Deprecate a method using `@Deprecated` and observe compiler warnings when used.
- **Mini Assignment:** Create a small interface and implementation; annotate methods properly and suppress an unchecked warning in a generics usage.
- **Common Mistakes:** Forgetting to annotate overridden methods (risk of silent bugs); misusing `@SuppressWarnings`.
- **Interview Questions:** "What does the `@Deprecated` annotation do?"
- **Quiz Questions:** "Which annotation is used to indicate that a method overrides a superclass method? (Answer: @Override)"

Chapter 17: Wrapper Classes and Autoboxing

- **Learning Objective:** Use wrapper classes for primitive types, understand autoboxing/unboxing, and utility methods of wrappers.
- **Skills Covered:** Converting between primitives and wrapper objects, using static methods like `Integer.parseInt`, and understanding performance implications.
- **Prerequisites:** Chapter 15 (Generics, since collections require objects).
- **Estimated Learning Time:** 2–3 hours.
- **Number of Lessons:** 1

*Lesson 17.1: **Wrappers, Autoboxing, and Unboxing***

- **Difficulty Level:** Beginner
- **Learning Outcome:** Use wrapper classes (`Integer`, `Double`, etc.) and understand automatic conversion between primitives and wrappers.
- **Topics Covered:**
- Wrapper classes and their constructors/static methods (`new Integer(5)`, `Integer.valueOf("5")`)
- Autoboxing/unboxing introduced in Java 5 (automatic conversion)
- Common methods: `parseInt`, `toString`, `valueOf`
- **Practical Exercises:**
- Store primitive ints in a `List<Integer>` (autoboxing example) and retrieve them.
- Convert a string to `Double` using `Double.parseDouble` and vice versa.

- **Mini Assignment:** Write a method that sums a `List<Integer>` by auto-unboxing each element.
- **Common Mistakes:** Comparing wrapper objects with == (should use `equals` for values outside -128 to 127); `NullPointerException` when unboxing a null.
- **Interview Questions:** "What is autoboxing and when was it introduced?" (Answer: Java 5)
- **Quiz Questions:** "What happens when you write `Integer x = 1000; Integer y = 1000; System.out.println(x == y);`? (Answer: false, because they are different objects beyond the integer cache range)"

Chapter 18: Date and Time API (java.time)

- **Learning Objective:** Work with Java's modern Date and Time API (`java.time` package) to handle dates, times, and durations.
- **Skills Covered:** Using `LocalDate`, `LocalTime`, `LocalDateTime`, formatting and parsing dates, and calculating durations/periods.
- **Prerequisites:** Chapter 17.
- **Estimated Learning Time:** 3–4 hours.
- **Number of Lessons:** 2

Lesson 18.1: ***LocalDate and LocalTime***

- **Difficulty Level:** Intermediate
- **Learning Outcome:** Create and manipulate `LocalDate`, `LocalTime`, and `LocalDateTime` objects, and format them for display.
- **Topics Covered:**
- `LocalDate.now()`, `LocalTime.now()`, `LocalDateTime.now()`
- Creating specific dates/times (`of(year, month, day)`)
- `plusDays()`, `minusHours()`, `getDayOfWeek()`, etc.
- Formatting with `DateTimeFormatter`
- **Practical Exercises:**
- Print the current date in format "dd/MM/yyyy".
- Calculate the date one week from today.
- **Mini Assignment:** Prompt user for a date of birth and compute the day of week they were born.
- **Common Mistakes:** Using the old `Date`/`Calendar` API instead of `java.time`; confusion between months (1–12 vs 0–11 in old API).
- **Interview Questions:** "Why is the new `java.time` API preferred over `java.util.Date`?" (Answer: Immutable and thread-safe)
- **Quiz Questions:** "Which class would you use for date without time? (Answer: LocalDate)"

Lesson 18.2: ***Duration and Period***

- **Difficulty Level:** Intermediate
- **Learning Outcome:** Compute the difference between dates and times using `Duration` (time-based) and `Period` (date-based).
- **Topics Covered:**
- `Duration.between(startTime, endTime)` for time differences
- `Period.between(startDate, endDate)` for date differences (years, months, days)
- Converting between units (e.g., hours from duration)
- **Practical Exercises:**
- Calculate how many days until a future event date entered by the user.
- Compute how many hours/minutes have passed since a given `LocalDateTime`.
- **Mini Assignment:** Determine a person's exact age (years, months, days) given their birthdate.
- **Common Mistakes:** Using `Duration` for dates (use `Period` instead); confusing units (duration is time-based).
- **Interview Questions:** "What is the difference between `Duration` and `Period`?"
- **Quiz Questions:** "How do you create a Duration of 90 minutes? (Answer: Duration.ofMinutes(90))"

Chapter 19: Lambda Expressions and Functional Interfaces

- **Learning Objective:** Write concise code using lambda expressions and functional interfaces introduced in Java 8.
- **Skills Covered:** Understand lambda syntax, use built-in functional interfaces (`Function`, `Consumer`, `Predicate`, `Supplier`), and write custom functional interfaces.
- **Prerequisites:** Chapter 14 (Collections) and Chapter 17 (Generics).
- **Estimated Learning Time:** 4–5 hours.
- **Number of Lessons:** 2

Lesson 19.1: ***Lambda Expressions Syntax***

- **Difficulty Level:** Intermediate
- **Learning Outcome:** Use lambda expressions to replace simple anonymous class implementations for single-method interfaces.
- **Topics Covered:**
- Syntax examples: `(parameters) -> expression` or `(params) -> { statements }`
- Target types: functional interfaces (e.g., `Runnable`, `Comparator`)
- Type inference in lambdas (no need to repeat types)
- **Practical Exercises:**
- Use a lambda to create a `Runnable` that prints a message when run.

- Sort an array of strings using `Arrays.sort` with a lambda comparator.
- **Mini Assignment:** Create a `Comparator<Integer>` with a lambda that sorts in descending order and test it.
- **Common Mistakes:** Missing type or parameter list (e.g., `(x,y)->x+y` vs `x,y->x+y`); confusion about `this` inside lambdas.
- **Interview Questions:** "What is a lambda expression in Java? Give an example."
- **Quiz Questions:** "Can a lambda throw a checked exception? (Answer: Only if the functional interface's method throws it.)"

Lesson 19.2: ***Common Functional Interfaces***

- **Difficulty Level:** Intermediate
- **Learning Outcome:** Use and implement functional interfaces from `java.util.function`: `Function`, `Consumer`, `Predicate`, `Supplier`.
- **Topics Covered:**
- `Function<T,R>` (T -> R), `Consumer<T>` (T -> void), `Predicate<T>` (T -> boolean), `Supplier<T>` (-> T)
- Method references (`ClassName::methodName`) as shorthand for lambdas
- Writing your own functional interface with `@FunctionalInterface`
- **Practical Exercises:**
- Use `Predicate<String>` to filter a list of strings for those starting with "A".
- Use `Supplier<Date>` to supply the current date/time when needed.
- **Mini Assignment:** Implement a method that takes a `List<String>` and a `Function<String, Integer>` to convert each string to its length, returning a list of lengths.
- **Common Mistakes:** Using lambdas where signature doesn't match functional interface; not using `@FunctionalInterface` annotation to get compile-time checks.
- **Interview Questions:** "Name two functional interfaces in `java.util.function`. What do they do?"
- **Quiz Questions:** "Which interface would you use for a function that takes no arguments and returns a value? (Answer: Supplier)"

Chapter 20: Streams API

- **Learning Objective:** Process collections of data declaratively using Java Streams for filtering, mapping, and reducing.
- **Skills Covered:** Converting collections to streams, using intermediate operations (`filter`, `map`, `sorted`), and terminal operations (`forEach`, `collect`, `reduce`).
- **Prerequisites:** Chapter 19 (Lambdas/Functional).
- **Estimated Learning Time:** 4–5 hours.
- **Number of Lessons:** 2

Lesson 20.1: ***Introduction to Streams***

- **Difficulty Level:** Intermediate
- **Learning Outcome:** Convert collections or arrays to streams and apply simple stream operations.
- **Topics Covered:**
- Obtaining streams: `collection.stream()`, `Stream.of(...)`
- Intermediate ops: `filter`, `map`, `flatMap`, `sorted`
- Terminal ops: `collect(toList())`, `forEach`, `count`, `reduce` basics
- **Practical Exercises:**
- Use a stream to filter a list of integers for even numbers and collect to a new list.
- Given a list of names, use a stream to create a list of their uppercase versions.
- **Mini Assignment:** Read a CSV file into a list of objects and use streams to process (e.g., filter rows by a condition).
- **Common Mistakes:** Reusing a stream after a terminal operation (illegal state); not consuming the stream.
- **Interview Questions:** "What does the `filter` operation do on a stream?"
- **Quiz Questions:** "Which method is terminal in a stream: `map()` or `collect()`? (Answer: collect)"

Lesson 20.2: ***Collectors and Aggregate Operations***

- **Difficulty Level:** Intermediate
- **Learning Outcome:** Perform aggregate operations like grouping, joining, and summarizing with `Collectors`.
- **Topics Covered:**
- `Collectors` utility: `toList()`, `toSet()`, `joining`, `groupingBy`, `partitioningBy`
- Primitive streams (`IntStream`, etc.) and `max`, `min`, `average`
- Parallel streams (mention concept)
- **Practical Exercises:**
- Use `Collectors.joining(", ")` to join a list of strings with commas.
- Use `groupingBy` to classify a list of strings by their length into a `Map<Integer, List<String>>`.
- **Mini Assignment:** Given a list of employee objects, use streams to group them by department and find the average salary per department.
- **Common Mistakes:** Collecting to wrong type; ignoring immutability of collected lists (unmodifiable).
- **Interview Questions:** "What is the difference between `findFirst()` and `findAny()` in streams?"
- **Quiz Questions:** "True or False: Streams are mutable and can store intermediate state. (Answer: False)"

Chapter 21: Multithreading (Introduction)

- **Learning Objective:** Understand threads in Java for concurrent execution, and the difference between platform and virtual threads.
- **Skills Covered:** Creating and running threads (via `Thread` class or `Runnable`), understanding thread lifecycle, basic synchronization.
- **Prerequisites:** Chapters 1–20.
- **Estimated Learning Time:** 5–7 hours.
- **Number of Lessons:** 3

*Lesson 21.1: **Creating Threads (Thread, Runnable)***

- **Difficulty Level:** Intermediate
- **Learning Outcome:** Create concurrent threads using the `Thread` class and `Runnable` interface.
- **Topics Covered:**
- Extending `Thread` vs implementing `Runnable`
- `start()` vs `run()` methods
- Thread states (new, runnable, blocked, etc.) and simple life-cycle
- **Practical Exercises:**
- Create two threads: one extending `Thread`, another implementing `Runnable`, each printing messages in a loop.
- Observe interleaving of outputs by running threads.
- **Mini Assignment:** Write a program that starts a background thread to print the current time every second while the main thread continues.
- **Common Mistakes:** Calling `run()` instead of `start()` (executes on main thread); forgetting synchronization when threads share data (conceptual).
- **Interview Questions:** "What is the difference between extending `Thread` and implementing `Runnable`?"
- **Quiz Questions:** "How do you start a thread in Java? (Answer: call its `start()` method)"

*Lesson 21.2: **Thread Methods and Lifecycle***

- **Difficulty Level:** Intermediate
- **Learning Outcome:** Use various `Thread` methods and handle interruptions.
- **Topics Covered:**
- `sleep()`, `join()`, `setPriority()` (mention), `isAlive()`
- Interrupting a thread (`interrupt()`, catching `InterruptedException`)
- Thread names (`getName()`, `setName()`) and daemon threads
- **Practical Exercises:**
- Create a thread that sleeps for 5 seconds and then prints a message; interrupt it from main thread and handle gracefully.

- Use `join()` to wait for a thread's completion before proceeding.
- **Mini Assignment:** Implement a countdown timer using a separate thread that can be stopped by user input (demonstrating `interrupt()`).
- **Common Mistakes:** Ignoring `InterruptedException`; assuming `stop()` method exists (deprecated).
- **Interview Questions:** "What does `Thread.sleep()` do, and why must we handle exceptions with it?"
- **Quiz Questions:** "Which method blocks the current thread until another thread finishes? (Answer: join())"

Lesson 21.3: ***Virtual Threads (Project Loom, Java 21)***

- **Difficulty Level:** Advanced Beginner (Introductory preview)
- **Learning Outcome:** Learn about virtual threads as lightweight threads and how to create them (using `Executors.newVirtualThreadPerTaskExecutor()`).
- **Topics Covered:**
- Concept of virtual threads (Java 21 feature)【4†L177-L185】
- Differences between platform threads and virtual threads (scalability)
- Creating and using virtual threads via the Executors API or `Thread.ofVirtual()`
- **Practical Exercises:**
- (If Java 21 is available) Create a virtual thread using `Thread.startVirtualThread(Runnable)` and observe behavior.
- Compare creating 1000 traditional threads vs 1000 virtual threads (conceptual observation).
- **Mini Assignment:** Rewrite the background clock program from 21.1 using a virtual thread.
- **Common Mistakes:** Confusing virtual threads with Thread subclasses; not using `java.util.concurrent` utilities properly.
- **Interview Questions:** "What is a virtual thread in Java, and how does it differ from a platform thread?"【4†L177-L185】
- **Quiz Questions:** "True or False: Virtual threads eliminate the need for synchronization. (Answer: False; synchronization is still needed)"

Chapter 22: Synchronization and Concurrency Control

- **Learning Objective:** Coordinate multiple threads accessing shared data using synchronization and locks to avoid race conditions.
- **Skills Covered:** Using `synchronized` methods/blocks, `volatile` keyword, and higher-level locks for thread safety.
- **Prerequisites:** Chapter 21 (Multithreading).
- **Estimated Learning Time:** 4–6 hours.

- **Number of Lessons:** 2

Lesson 22.1: ***Synchronized Methods and Blocks***

- **Difficulty Level:** Intermediate
- **Learning Outcome:** Prevent concurrent access issues by synchronizing code sections.
- **Topics Covered:**
- `synchronized` keyword on methods and blocks
- Locking on `this` vs on other objects
- Concept of race conditions (incrementing a counter unsafely)
- **Practical Exercises:**
- Write a multithreaded program where two threads increment a shared counter without synchronization (demonstrate incorrect result), then fix with `synchronized`.
- Use a synchronized block locking on a private object.
- **Mini Assignment:** Implement a simple thread-safe class (e.g., a counter) using synchronization.
- **Common Mistakes:** Synchronizing on public objects (risk external interference); forgetting to synchronize leading to inconsistent state.
- **Interview Questions:** "What is the difference between a synchronized method and a synchronized block?"
- **Quiz Questions:** "Which keyword ensures that only one thread can execute a method at a time? (Answer: synchronized)"

Lesson 22.2: ***Locks and Volatile Variables***

- **Difficulty Level:** Intermediate
- **Learning Outcome:** Use explicit locks (`ReentrantLock`) and the `volatile` keyword for advanced concurrency control.
- **Topics Covered:**
- `volatile` for ensuring visibility of changes across threads (for primitive read/write)
- Using `ReentrantLock` and `Lock` interface for manual locking
- `Condition` objects for wait/notify patterns (brief mention)
- **Practical Exercises:**
- Mark a shared boolean flag as `volatile` and observe behavior in threads (e.g., stopping a loop).
- Use `ReentrantLock` instead of `synchronized` to protect a critical section.
- **Mini Assignment:** Rewrite the synchronized counter from 22.1 using `ReentrantLock`.
- **Common Mistakes:** Assuming `volatile` provides atomicity (it does not); forgetting to `unlock()` in `finally` block.
- **Interview Questions:** "When should you use `volatile` instead of `synchronized`?"

- **Quiz Questions:** "True or False: Declaring a variable as volatile makes operations on it atomic. (Answer: False)"

Chapter 23: Executor Framework (Thread Pools)

- **Learning Objective:** Manage groups of threads efficiently using the Executor framework (thread pools).
- **Skills Covered:** Using `ExecutorService`, `Executors` utility class, submitting `Callable` tasks, and shutting down executors.
- **Prerequisites:** Chapter 21 (Multithreading).
- **Estimated Learning Time:** 3–4 hours.
- **Number of Lessons:** 2

*Lesson 23.1: **ExecutorService and Fixed Thread Pool***

- **Difficulty Level:** Intermediate
- **Learning Outcome:** Submit tasks to a thread pool using `ExecutorService` instead of managing threads manually.
- **Topics Covered:**
- Creating thread pools: `Executors.newFixedThreadPool(n)`
- Submitting `Runnable` tasks with `execute()` or `submit()`
- Shutting down the executor (`shutdown()`, `shutdownNow()`)
- **Practical Exercises:**
- Create a fixed thread pool of size 3 and submit 10 tasks that print their thread name and sleep.
- Observe that only 3 threads are created, reusing threads for tasks.
- **Mini Assignment:** Convert a previously made multi-threaded program to use `ExecutorService` instead of manually starting threads.
- **Common Mistakes:** Forgetting to shutdown (program hangs); submitting non-terminating tasks.
- **Interview Questions:** "Why use a thread pool instead of creating threads manually?"
- **Quiz Questions:** "How do you properly shut down an ExecutorService after use? (Answer: call shutdown())"

*Lesson 23.2: **Callable and Futures***

- **Difficulty Level:** Intermediate
- **Learning Outcome:** Use `Callable` for tasks that return results and retrieve those results via `Future`.
- **Topics Covered:**
- `Callable<V>` interface (similar to `Runnable` but returns value)
- `Future<V>` and methods `get()`, `isDone()`, `cancel()`

- `submit()` vs `execute()` difference (submit returns Future)
- **Practical Exercises:**
- Submit a `Callable<Integer>` that computes the sum of numbers 1–N and retrieve the result with `Future.get()`.
- Use `invokeAll()` on a list of Callables to run in parallel and get a list of Futures.
- **Mini Assignment:** Implement a parallel prime checker: submit callables for checking prime numbers and collect results.
- **Common Mistakes:** Calling `get()` without checking completion (blocks indefinitely); not handling `ExecutionException`.
- **Interview Questions:** "What is the difference between `submit` and `execute` in ExecutorService?"
- **Quiz Questions:** "Which interface allows a task to return a result? (Answer: Callable)"

Chapter 24: JDBC (Database Connectivity)

- **Learning Objective:** Connect to relational databases and perform SQL operations using JDBC.
- **Skills Covered:** Registering JDBC driver, establishing `Connection`, creating `Statement`/`PreparedStatement`, executing queries/updates, and processing `ResultSet`.
- **Prerequisites:** Chapters 1–23, and basic SQL knowledge.
- **Estimated Learning Time:** 4–5 hours.
- **Number of Lessons:** 2

*Lesson 24.1: **JDBC Basics – Connecting and Querying***

- **Difficulty Level:** Intermediate
- **Learning Outcome:** Connect to a database and execute simple SQL queries.
- **Topics Covered:**
- Loading JDBC driver (modern DriverManager handles automatically)
- `Connection` object via `DriverManager.getConnection(url, user, pass)`
- Using `Statement` to execute `SELECT` queries and iterate over `ResultSet`
- **Practical Exercises:**
- Connect to a sample database (e.g., SQLite or MySQL), query a table of users, and print results.
- Handle `SQLException` and ensure `Connection`/`Statement` are closed in `finally` or try-with-resources.
- **Mini Assignment:** Create a Java program that inserts a new row into a database table using JDBC.

- **Common Mistakes:** Using `executeQuery` for update statements; forgetting to close connections (resource leak).
- **Interview Questions:** "What JDBC objects do you use to execute SQL queries?" (Answer: Connection, Statement, ResultSet).
- **Quiz Questions:** "Which method is used to execute a SELECT statement? (Answer: executeQuery)"

Lesson 24.2: ***PreparedStatement and Transactions***

- **Difficulty Level:** Intermediate
- **Learning Outcome:** Use `PreparedStatement` to safely execute parameterized SQL and manage transactions.
- **Topics Covered:**
- `PreparedStatement` advantages (prevent SQL injection, reuse, parameters)
- Setting parameters (`setString`, `setInt`, etc.) and executing (`executeQuery` / `executeUpdate`)
- Transactions: `setAutoCommit(false)`, `commit()`, `rollback()`
- **Practical Exercises:**
- Write a `PreparedStatement` to search for records with a certain criterion (e.g., WHERE id = ?).
- Perform multiple updates in one transaction and roll back on error.
- **Mini Assignment:** Build a simple login check using JDBC: prompt user for username/password, use `PreparedStatement` to validate against a `users` table.
- **Common Mistakes:** Concatenating user input directly into SQL (risking injection); not handling `SQLException` properly.
- **Interview Questions:** "Why use PreparedStatement over Statement?"
- **Quiz Questions:** "How do you disable auto-commit in JDBC? (Answer: connection.setAutoCommit(false))"

Chapter 25: Java Best Practices and Code Quality

- **Learning Objective:** Learn industry best practices in Java coding for readability, maintainability, and efficiency.
- **Skills Covered:** Applying naming conventions, commenting and documentation, error handling strategies, and coding style consistency.
- **Prerequisites:** All previous chapters.
- **Estimated Learning Time:** 3–4 hours.
- **Number of Lessons:** 1

Lesson 25.1: ***Java Coding Standards and Best Practices***

- **Difficulty Level:** Intermediate

- **Learning Outcome:** Follow standard coding guidelines (naming, formatting) and apply best practices (e.g., DRY, KISS, handling exceptions properly).
- **Topics Covered:**
- Naming conventions recap (classes, methods, variables, constants)
- Formatting: indentation (4 spaces), braces position, line length (e.g., 100 chars)【26†L40-L49】
- Commenting: when to comment (explain why, not what) and JavaDoc usage
- Exception handling: avoid empty catch blocks, prefer specific exceptions, clean up resources
- **Practical Exercises:**
- Refactor a messy piece of code to follow Java naming and formatting conventions (see Google Java Style Guide【26†L42-L50】).
- Review a code snippet and identify violations of coding best practices.
- **Mini Assignment:** Establish a small team project guideline document incorporating best practices learned (like code reviews checklist).
- **Common Mistakes:** Magic numbers (use constants instead), long methods (need to be refactored), repeated code blocks (not DRY).
- **Interview Questions:** "What are some Java naming conventions for constants and variables?"
- **Quiz Questions:** "According to Google Java Style, what is the maximum recommended line length? (Answer: 100 characters)【26†L40-L49】"

Chapter 26: Design Principles and Patterns (Overview)

- **Learning Objective:** Gain awareness of fundamental design principles (SOLID, DRY, etc.) and common design patterns for building robust Java applications.
- **Skills Covered:** Understanding SOLID principles, applying DRY/KISS/YAGNI, and recognizing patterns like Singleton, Factory, Observer at a basic level.
- **Prerequisites:** Chapter 7–9 (OOP concepts).
- **Estimated Learning Time:** 3–4 hours.
- **Number of Lessons:** 1

Lesson 26.1: ***SOLID Principles and Common Patterns***

- **Difficulty Level:** Intermediate
- **Learning Outcome:** Understand high-level object-oriented design principles and when to apply design patterns.
- **Topics Covered:**
- SOLID principles: Single Responsibility, Open/Closed, Liskov, Interface Segregation, Dependency Inversion
- DRY (Don't Repeat Yourself), KISS (Keep It Simple)

- Brief introductions to patterns: Singleton, Factory, Strategy, Observer (conceptual, more depth in advanced courses)
- **Practical Exercises:**
- Analyze a sample class violating Single Responsibility (e.g., one class doing file I/O and data processing) and refactor.
- Recognize a scenario where a Factory pattern might be useful (e.g., creating objects based on input).
- **Mini Assignment:** Design a class structure for a notification system using Observer pattern (just a high-level sketch).
- **Common Mistakes:** Overusing patterns (making simple problems complicated); tight coupling between classes violating dependency inversion.
- **Interview Questions:** "Explain the Single Responsibility Principle."
- **Quiz Questions:** "Which principle suggests classes should depend on abstractions, not concretions? (Answer: Dependency Inversion Principle)"

Chapter 27: Debugging Techniques

- **Learning Objective:** Learn how to find and fix errors in Java programs using IDE debuggers and logging.
- **Skills Covered:** Setting breakpoints, stepping through code, inspecting variables, and using logging frameworks (`java.util.logging` or `Log4j`).
- **Prerequisites:** Chapters 1–26.
- **Estimated Learning Time:** 2–3 hours.
- **Number of Lessons:** 1

*Lesson 27.1: **Using Debugger and Logging***

- **Difficulty Level:** Intermediate
- **Learning Outcome:** Use IDE debugging tools (stepping, watches) and add logging statements for troubleshooting.
- **Topics Covered:**
- Setting breakpoints in IntelliJ/Eclipse, stepping over/into methods, inspecting call stack and variable values
- Conditional breakpoints and watches
- Using `System.out.println` vs a logging framework (`Logger`)
- Logging levels (INFO, DEBUG, ERROR) and best practices for logging
- **Practical Exercises:**
- Debug a buggy loop by setting a breakpoint and stepping through iterations.
- Integrate simple logging into an application (using `Logger`) and control log output via configuration.

- **Mini Assignment:** Take a previously written program with bugs and demonstrate using the debugger to find the bug.
- **Common Mistakes:** Not reading stack traces properly; commenting out code instead of fixing the root cause.
- **Interview Questions:** "What is the difference between a breakpoint and a watch in debugging?"
- **Quiz Questions:** "True or False: You should leave debugging `System.out.println` statements in production code. (Answer: False)"

Chapter 28: Unit Testing Basics

- **Learning Objective:** Write and run automated tests using JUnit to verify code correctness.
- **Skills Covered:** Creating test cases, using assertions, running tests, and understanding the testing lifecycle.
- **Prerequisites:** Chapters 1–27.
- **Estimated Learning Time:** 3–4 hours.
- **Number of Lessons:** 1

*Lesson 28.1: **JUnit Testing Framework***

- **Difficulty Level:** Intermediate
- **Learning Outcome:** Create JUnit test classes and methods, and use assertions to validate expected results.
- **Topics Covered:**
- Setting up JUnit (version 5) in project (Maven/Gradle or IDE)
- Writing test methods annotated with `@Test`
- Common assertions (`assertEquals`, `assertTrue`, `assertThrows`)
- Test fixtures (`@BeforeEach`, `@AfterEach`)
- **Practical Exercises:**
- Write a test for a simple method (e.g., sum of two numbers) and run it.
- Test exception throwing by using `assertThrows`.
- **Mini Assignment:** Develop unit tests for a `Calculator` class (methods add, subtract, divide) covering normal and edge cases (e.g., division by zero).
- **Common Mistakes:** Not using assertions properly; tests that depend on environment (not repeatable).
- **Interview Questions:** "What is the purpose of a unit test?"
- **Quiz Questions:** "Which annotation marks a test method in JUnit? (Answer: @Test)"

Chapter 29: Version Control with Git for Java Developers

- **Learning Objective:** Manage source code using Git and GitHub, understanding basic commands and workflows.
- **Skills Covered:** Repository creation, commits, branching, merging, and collaborating using Git.
- **Prerequisites:** None specific (development environment setup).
- **Estimated Learning Time:** 3–4 hours.
- **Number of Lessons:** 1

Lesson 29.1: ***Git Essentials***

- **Difficulty Level:** Beginner
- **Learning Outcome:** Use Git to track code changes, create branches, and collaborate on Java projects.
- **Topics Covered:**
- Git installation and initialization (`git init`), staging (`git add`), commits (`git commit`)
- Branching (`git branch`, `git checkout -b`), merging (`git merge`)
- Remote repositories: connecting to GitHub, pushing (`git push`) and pulling (`git pull`)
- **Practical Exercises:**
- Initialize a Git repo in a Java project folder, make a few commits.
- Create a new branch, modify code, merge back into `main`.
- **Mini Assignment:** Set up a GitHub repository for a course project and push your local Java code to it.
- **Common Mistakes:** Committing sensitive data; merge conflicts.
- **Interview Questions:** "What is the difference between `git merge` and `git rebase`? (High-level answer expected.)"
- **Quiz Questions:** "Which command stages a file for commit? (Answer: git add)"

Chapter 30: Build Tools – Maven and Gradle Introduction

- **Learning Objective:** Automate project build processes using Maven or Gradle.
- **Skills Covered:** Project structure, dependencies management, building and running Java applications.
- **Prerequisites:** Chapter 29 (Git) and general Java knowledge.
- **Estimated Learning Time:** 2–3 hours.
- **Number of Lessons:** 1

Lesson 30.1: ***Maven Basics (and Intro to Gradle)***

- **Difficulty Level:** Intermediate

- **Learning Outcome:** Create a Maven project, add dependencies, and build with `mvn`. Optionally understand Gradle's equivalent (Groovy/Kotlin DSL).
- **Topics Covered:**
- Maven `pom.xml` structure (groupId, artifactId, version)
- Declaring dependencies (e.g., JUnit, JSON library)
- Running Maven goals: `compile`, `test`, `package`
- Brief mention of Gradle init and `build.gradle` (without deep dive)
- **Practical Exercises:**
- Convert a Java project into a Maven project using an IDE or `mvn archetype`.
- Add a dependency (e.g., Google Gson) to `pom.xml` and use it in code.
- **Mini Assignment:** Create a Maven project for one of the mini projects (below) and include JUnit and logging dependencies.
- **Common Mistakes:** Forgetting to run `mvn install` to install dependencies; conflicts in dependency versions.
- **Interview Questions:** "What is a Maven dependency and where do you specify it?"
- **Quiz Questions:** "Which file defines a Maven project's dependencies? (Answer: pom.xml)"

Chapter 31: Real-World Project Development

- **Learning Objective:** Apply accumulated knowledge to plan and develop a Java project following industry practices (from requirements to deployment).
- **Skills Covered:** Requirement analysis, project setup, iterative development, documentation, version control in practice.
- **Prerequisites:** Chapters 1–30.
- **Estimated Learning Time:** 5–6 hours.
- **Number of Lessons:** 2

*Lesson 31.1: **Project Planning and Setup***

- **Difficulty Level:** Intermediate
- **Learning Outcome:** Set up a new Java project by defining requirements, choosing libraries/frameworks, and configuring the build.
- **Topics Covered:**
- Defining project scope and user stories
- Choosing between Maven/Gradle, setting up repository structure
- Creating package layout and main classes according to features
- **Practical Exercises:**
- Outline a small project's requirements (e.g., library management system) and break into tasks.

- Initialize a Git repository and create initial project files (with `.gitignore` for Java).
- **Mini Assignment:** For a given project idea (or one from below), write a README with features and setup instructions.
- **Common Mistakes:** Poor project structure, missing documentation, setting up too many or no dependencies.
- **Interview Questions:** "What is included in a good README for a Java project?"
- **Quiz Questions:** "True or False: All Java projects should start with creating the main method. (Answer: False; focus on planning first)"

Lesson 31.2: ***Iterative Development and Versioning***

- **Difficulty Level:** Intermediate
- **Learning Outcome:** Develop the project incrementally, using Git for version control and ensuring code quality through testing.
- **Topics Covered:**
- Agile/iterative workflow: small increments, feature branches, code reviews
- Unit testing during development (Test-Driven Development introduction)
- Continuous Integration (CI) basics (mention tools like GitHub Actions)
- **Practical Exercises:**
- Implement a core feature of the project (e.g., user login or data entry form) and write tests for it.
- Use Git branching workflow: create a branch for a new feature, merge when complete.
- **Mini Assignment:** Set up a simple GitHub Actions workflow that compiles and tests the project on each push.
- **Common Mistakes:** Trying to do too much at once (no small commits), skipping tests, working on `main` without branches.
- **Interview Questions:** "What is continuous integration and why is it important?"
- **Quiz Questions:** "Which Git workflow would you use to add a new feature without affecting the main code? (Answer: Feature branch)"

Chapter 32: Interview Preparation

- **Learning Objective:** Prepare for Java developer interviews by reviewing common questions, coding problems, and soft skills tips.
- **Skills Covered:** Core Java Q&A, coding problem practice, communication of solutions.
- **Prerequisites:** All technical chapters.
- **Estimated Learning Time:** 3–5 hours.
- **Number of Lessons:** 1

Lesson 32.1: ***Core Java Interview Questions***

- **Difficulty Level:** Beginner/Intermediate (Fresher level)
- **Learning Outcome:** Recall and articulate answers to frequently asked Java interview questions; practice explaining concepts clearly.
- **Topics Covered:**
- Review of fundamentals: OOP concepts, Java memory model, collections, exception handling, Java 8 features, etc.
- Behavioral/soft questions (e.g., project experience, problem-solving approach)
- Coding challenge strategies (whiteboard/mock coding)
- **Practical Exercises:**
- Practice answering 10 common Java interview questions (self or mock interview).
- Solve a simple coding problem on an online judge (e.g., reverse a string, find duplicates).
- **Mini Assignment:** Compile a list of your strengths and experiences related to Java projects and prepare elevator pitch.
- **Common Mistakes:** Memorizing answers without understanding; poor communication of thought process.
- **Interview Questions:** [This lesson is about answering questions, not providing Qs, so skip.]
- **Quiz Questions:** [Not applicable; this is preparation lesson.]

Chapter 33: Final Capstone Project

- **Learning Objective:** Demonstrate comprehensive Java skills by designing and implementing a complex, real-world application.
- **Skills Covered:** Full lifecycle development: design, coding, testing, and presentation of a complete software product.
- **Prerequisites:** Completion of all course chapters.
- **Estimated Learning Time:** 10+ hours (suggested as a final project effort).
- **Number of Lessons:** 1

Lesson 33.1: ***Capstone Project Development***

- **Difficulty Level:** Intermediate/Advanced (Project-based)
- **Learning Outcome:** Independently develop a major Java application (e.g., Inventory Management, E-commerce backend) using principles and tools learned.
- **Topics Covered:**
- Defining project scope and requirements based on a real-world scenario
- Applying OOP, collections, file I/O/DB, multithreading, etc., as relevant
- Using build tools, version control, testing, and documentation in the project
- **Practical Exercises:**

- Plan and implement major features of the capstone project (at least 3-5 key functionalities)
- Demonstrate the project (running demo, explaining code architecture)
- **Mini Assignment:** Write a final report or presentation covering project architecture, challenges, and future improvements.
- **Common Mistakes:** Overreaching scope (trying to do too much); insufficient testing and documentation.
- **Interview Questions:** [Not applicable; this is project execution.]
- **Quiz Questions:** [Not applicable.]

Additional Resources and Roadmap

- **Complete Roadmap:** A structured path from Java fundamentals to advanced topics, aligning with the chapters above. Students can follow this sequential roadmap to build knowledge step-by-step.
- **Beginner-to-Job-Ready Path:** Suggested timelines and milestones (e.g., 3 months practice plan) covering daily/weekly goals to transition from learning to employable skills.
- **Recommended Practice Strategy:** Tips on daily coding practice, using spaced repetition for concepts, pair programming, and contributing to open source for hands-on learning.
- **Java Coding Standards:** Summary of best practices (naming, formatting) and references to style guides (e.g., Google Java Style Guide【26†L42-L50】).
- **Coding Challenges:** Curated list of practice problems (from easy to hard) on arrays, OOP, algorithms, and streams to reinforce each topic.
- **200+ Practice Questions:** A bank of multiple-choice and short-answer questions covering all chapters (Java basics to streams) for self-assessment.
- **100+ Interview Questions:** Frequently asked interview questions with answers, covering core Java, OOP, collections, concurrency, JDBC, and more (e.g., as compiled in GeeksforGeeks【20†L41-L49】).
- **50 Mini Projects:** Project ideas to build progressively, such as calculator, address book, simple game, file parser, REST API client, etc., to apply learned concepts in small-scale applications.
- **10 Major Projects:** Larger project suggestions (e.g., Library Management System, E-commerce Backend, Chat Application, Quiz Platform, JavaFX GUI app) to prepare portfolio-worthy work.
- **Best Resources:**
- **Books:** *Effective Java*, *Head First Java*, *Java: The Complete Reference*, *Java Concurrency in Practice*.

- **Documentation:** Oracle's official Java tutorials and API docs (e.g., Java SE documentation【3†L124-L132】).
- **GitHub Repositories:** Curated Java course repos and example projects (e.g., `iluwatar/java-design-patterns`, `TheAlgorithms/Java`).
- **Coding Platforms:** LeetCode, HackerRank, CodeChef, CodeSignal (for algorithm practice), and Baeldung or Oracle tutorials for reference.
